

Introduction to Dynamic Programming (and Also on Trees)



The Core Idea



What is Dynamic Programming?

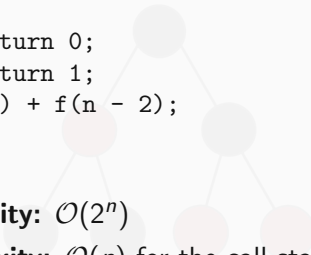
- The essence of dynamic programming is to **avoid repeated calculation**.
- Many DP problems are naturally solvable by recursion, DP only optimises those solutions.
- **Key Insight:** Identify overlapping subproblems and store their results in a lookup table.
- Two main approaches:
 - **Top-Down (Memoization):** Write the recursive solution, then cache results.
 - **Bottom-Up (Tabulation):** Build the solution iteratively from base cases upward, with a *state* transition.

The Fibonacci Sequence

- One of the most classic examples of dynamic programming.
- Recursive formulation: $f(n) = f(n-1) + f(n-2)$ for $n \geq 2$.
- Base cases: $f(0) = 0$, $f(1) = 1$.
- The naive recursive approach has exponential time complexity.
- **Why?** Each call to $f(n)$ spawns two similarly-sized subproblems, giving approximately $\mathcal{O}(2^n)$ total calls.
- **Example:** Computing $f(29)$ with naive recursion results in *over 1 million* function calls.

Naive Recursive Fibonacci

```
int f(int n) {  
    if (n == 0) return 0;  
    if (n == 1) return 1;  
    return f(n - 1) + f(n - 2);  
}
```



- **Time Complexity:** $\mathcal{O}(2^n)$
- **Space Complexity:** $\mathcal{O}(n)$ for the call stack.
- Unusable for large inputs. We need a better approach.

Top-Down DP with Memoization



The Memoization Principle

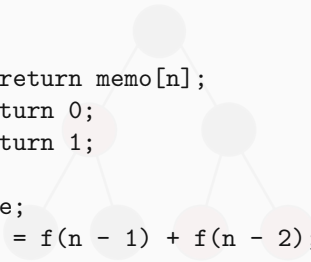
- **Key Observation:** The number of *distinct* subproblems needed to compute $f(n)$ is only $\mathcal{O}(n)$.
- We only ever need $f(0), f(1), \dots, f(n)$.
- **The Fix:** Solve each subproblem exactly once and store the result in a lookup table.
- Subsequent calls for the same value return the cached result in $\mathcal{O}(1)$.
- **Result:** Total time complexity drops from $\mathcal{O}(2^n)$ to $\mathcal{O}(n)$.
- **Impact:** $f(29)$ now requires only *57 function calls* down from a million.

Memoized Fibonacci (Array)

```
const int MAXN = 100;
bool found[MAXN];
int memo[MAXN];

int f(int n) {
    if (found[n]) return memo[n];
    if (n == 0) return 0;
    if (n == 1) return 1;

    found[n] = true;
    return memo[n] = f(n - 1) + f(n - 2);
}
```



- Arrays give $\mathcal{O}(1)$ lookup with minimal overhead — preferred when possible.
- **Caveat:** $f(46)$ is the last Fibonacci number that fits in a signed 32-bit integer.

Runtime Analysis of Memoization

→ General Rule:

Total Work = Work per Sub-problem $\times$ Number of Sub-problems

→ **Array-based memo:** $\mathcal{O}(1)$ lookup $\times \mathcal{O}(n)$ subproblems
= $\mathcal{O}(n)$.

→ `std::unordered_map`: $\mathcal{O}(1)$ average lookup = $\mathcal{O}(n)$ average overall.

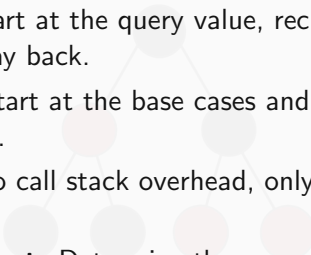
→ `std::map`: $\mathcal{O}(\log n)$ lookup $\times \mathcal{O}(n)$ subproblems
= $\mathcal{O}(n \log n)$.

→ **Recommendation:** Use arrays whenever the state fits. Use maps only when the state involves complex types like vectors or strings.

Bottom-Up DP



The Bottom-Up Approach

- 
- **Top-Down:** Start at the query value, recurse to base cases, cache on the way back.
 - **Bottom-Up:** Start at the base cases and iteratively extend to the target value.
 - No recursion, no call stack overhead, only iterative, so generally faster.
 - **Key Requirement:** Determine the correct filling order so that all dependencies are resolved before they are needed.

Bottom-Up Fibonacci

```
const int MAXN = 100;
int fib[MAXN];

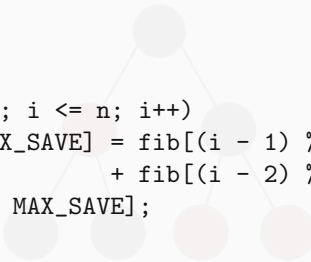
int f(int n) {
    fib[0] = 0;
    fib[1] = 1;
    for (int i = 2; i <= n; i++)
        fib[i] = fib[i - 1] + fib[i - 2];
    return fib[n];
}
```

- **Time Complexity:** $\mathcal{O}(n)$.
- **Space Complexity:** $\mathcal{O}(n)$ but can we do a bit better?

Optimizing Space to $\mathcal{O}(1)$

```
const int MAX_SAVE = 3;
int fib[MAX_SAVE];

int f(int n) {
    fib[0] = 0;
    fib[1] = 1;
    for (int i = 2; i <= n; i++)
        fib[i % MAX_SAVE] = fib[(i - 1) % MAX_SAVE]
            + fib[(i - 2) % MAX_SAVE];
    return fib[n % MAX_SAVE];
}
```

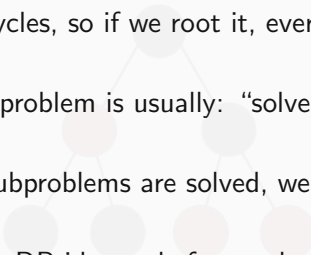


- Only the last two values are ever needed, so we keep a rolling window of size 3.
- The modulo operator is a common trick for maintaining only the necessary window.
- **Space Complexity:** $\mathcal{O}(1)$

Dynamic Programming on Trees



Why Tree DP?

- 
- A tree has no cycles, so if we root it, every node has a clear set of children.
 - The natural subproblem is usually: “solve the subtree rooted at this node.”
 - Once all child subproblems are solved, we can merge them at the parent.
 - This is the same DP idea as before: solve smaller pieces and combine the answers.

AtCoder DP Contest Problem P, "Independent Set"

https://atcoder.jp/contests/dp/tasks/dp_p

- We are given a tree with N vertices.
- Paint each vertex either **white** or **black**.
- No edge may have **both endpoints black**.
- Count the number of valid colourings modulo $10^9 + 7$
(reminder to a couple weeks back with modular arithmetic with our guy Yersin!).
- Since $N \leq 10^5$, we need something much faster than brute force.

Things to See

- The graph is a **tree**, so if we root it, every node gets a set of children.
- Subtrees of different children do not overlap.
- The constraint is **local**: only adjacent vertices matter.
- So once the colour of a node is fixed, each child subtree can be solved independently.

So What is DP on Trees?

→ Instead of something like this for bottom up:

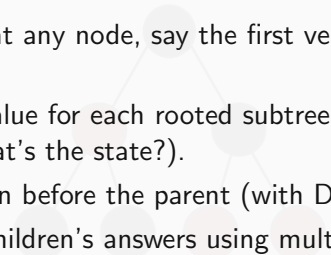
$dp[i] = \max(A[i] + dp[i + 2], dp[i + 1])$

→ We have this:

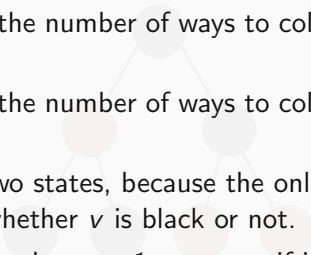
$dp[i] = \max(A[i] + \text{sum_grandchild}[i + 1], \text{SUM child}[i])$

→ This “solve a rooted subtree from its children” pattern is what DP on trees means.

Okay, so, What to Do?

- 
- Root the tree at any node, say the first vertex, zero (if 0-indexed).
 - Define a DP value for each rooted subtree (what's the transition? what's the state?).
 - Process children before the parent (with DFS).
 - Combine the children's answers using multiplication, because the choices are independent.

What's the State?

- 
- Let $dp[v][0]$ be the number of ways to colour the subtree of v , if v is **white**.
 - Let $dp[v][1]$ be the number of ways to colour the subtree of v , if v is **black**.
 - We only need two states, because the only relevant information is whether v is black or not.
 - For a leaf, both values are 1: one way if it is white, one way if it is black.

What's the Recurrence/Transition?

→ If v is black, then every child must be white:

$$dp[v][1] = \prod_{u \in \text{child}(v)} dp[u][0]$$

→ If v is white, then each child can be white or black:

$$dp[v][0] = \prod_{u \in \text{child}(v)} (dp[u][0] + dp[u][1])$$

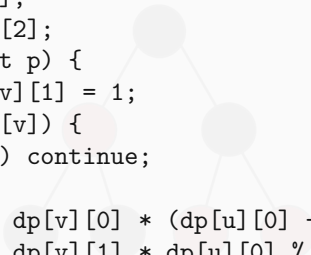
→ Why a product? Because the child subtrees are independent.

→ Final answer:

$$(dp[\text{root}][0] + dp[\text{root}][1]) \bmod (10^9 + 7)$$

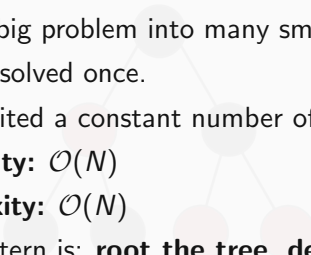
DFS Post-Order Implementation

```
const long long MOD = 1000000007LL;
const int MAXN = 100000 + 5;
vector<int> g[MAXN];
long long dp[MAXN][2];
void dfs(int v, int p) {
    dp[v][0] = dp[v][1] = 1;
    for (int u : g[v]) {
        if (u == p) continue;
        dfs(u, v);
        dp[v][0] = dp[v][0] * (dp[u][0] + dp[u][1]) % MOD;
        dp[v][1] = dp[v][1] * dp[u][0] % MOD;
    }
}
```

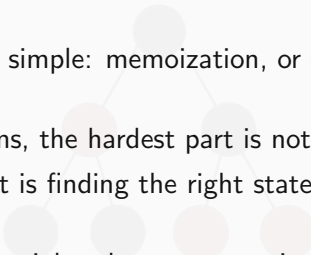


- Call `dfs(root, 0)` after building the adjacency list.
- The answer is $(dp[root][0] + dp[root][1]) \% MOD$.

Why This Is Tree DP

- 
- We turned one big problem into many small subtree problems.
 - Each subtree is solved once.
 - Each edge is visited a constant number of times.
 - **Time complexity:** $\mathcal{O}(N)$
 - **Space complexity:** $\mathcal{O}(N)$
 - The general pattern is: **root the tree, define subtree states, recurse, combine.**

Some Notes on Dynamic Programming

- 
- One of the most important tricks. It is often the difference between a solution that is impossible and one that runs in time.
 - The core idea is simple: memoization, or define the states and transitions
 - In many problems, the hardest part is not the transition itself.
 - The hardest part is finding the right state to describe the problem.
 - Once the state is right, the recurrence is often surprisingly simple.
 - So whenever you see recursion and overlapping structure, ask:

“Can I cache this?” “What’s the state?”